

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I Y SRAVAN KUMAR (192472289) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SRAVAN Y

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task

1. Hadoop Job Fails Due to Uneven Input Splits and Idle Reducers

Problem

The Hadoop job repeatedly fails because the input data is divided unevenly. Some mapper tasks receive very large amounts of data while others receive smaller amounts. As a result, some reducers remain idle.

Likely Design Error

The main error is **poor input partitioning and data distribution**. Large or uneven input splits create a workload imbalance. This is also called **data skew**.

Corrective Actions

- Use appropriate HDFS block/input split sizes.
- Check whether some files are much larger than others.
- Use better input partitioning.
- Balance the mapper workload.
- Use a **Combiner** to reduce intermediate data.
- Review the number of reducers based on data volume.
- Use a suitable partitioner when certain keys contain much more data than others.
- Monitor slow mapper and reducer tasks.

Result

Balanced input splits allow mapper tasks to finish more evenly and provide better work distribution to reducers.

Conclusion

The problem is mainly caused by **uneven data partitioning/data skew**. Proper input splitting, partitioning, and reducer configuration can improve Hadoop job performance.

2. HDFS Data Unavailability After a Single Node Failure

Problem

An HDFS deployment experiences data unavailability even when only one DataNode fails. This indicates that the system does not have sufficient fault tolerance.

Likely Design Errors

1. Low Replication Factor

If important blocks have only one copy, failure of that DataNode makes the data unavailable.

2. Poor Replica Placement

If all replicas are placed on the same rack or failure domain, a rack failure can affect multiple copies.

3. NameNode Design Problem

If the deployment has only one NameNode and it fails, the HDFS namespace may become unavailable.

Corrective Actions

- Use an appropriate replication factor, commonly **3** for important HDFS data.
- Distribute replicas across different DataNodes.
- Use **rack awareness**.
- Configure **NameNode High Availability** with Active and Standby NameNodes.
- Monitor under-replicated blocks.
- Regularly check cluster health.

Conclusion

The probable problem is **insufficient replication or lack of NameNode high availability**. Proper replication, rack-aware placement, and NameNode HA improve HDFS availability after node failures.

3. Duplicate Records from ETL and Apache NiFi

Problem

An ETL and Apache NiFi ingestion workflow is creating duplicate records in the analytics store.

Likely Causes

1. Repeated Data Ingestion

The same source records may be processed more than once.

2. Missing Unique Identifier

There may be no reliable ID to identify whether a record has already been inserted.

3. Pipeline Retry

If NiFi retries a failed operation after the destination actually received the record, the same record may be inserted again.

4. Incorrect ETL Logic

A join or transformation may accidentally generate multiple copies.

5. Lack of Deduplication

The analytics database may accept duplicate records without checking them.

Corrective Actions

- Assign a **unique record ID**.
- Use incremental or change-based data ingestion.
- Maintain ingestion checkpoints.
- Use NiFi flow-file attributes to track processing.
- Apply deduplication before loading data.
- Use database **primary keys/unique constraints**.
- Design the pipeline to support **idempotent processing**, so processing the same event again does not create another record.
- Check retry and failure-handling logic.

Example

Without deduplication:

Customer 101 → Insert

Customer 101 → Insert again

With a unique ID:

Customer 101 → Already exists → Do not insert again

Conclusion

The main issue is usually **lack of unique identification and duplicate-control mechanisms**. Idempotent processing, checkpoints, and database constraints can prevent duplicate records.

4. YARN Resource Starvation with Multiple Users

Problem

A YARN cluster experiences resource starvation when many users submit jobs simultaneously. Some jobs consume most of the available CPU and memory, leaving insufficient resources for other users.

Likely Scheduling Error

The cluster probably has:

- Poor queue configuration
- No resource limits
- Incorrect capacity allocation
- No fair scheduling policy
- Excessive resource requests from individual jobs

Corrective Actions

1. Configure Queues

Create separate queues for different workloads.

Example:

ETL Queue → 40%

Reporting Queue → 30%

ML Queue → 30%

2. Use Fair Resource Sharing

Use a suitable YARN scheduling policy to distribute resources fairly among users or applications.

3. Set Resource Limits

Set:

- Minimum resources
- Maximum resources
- Per-user limits
- Application limits

4. Set Priorities

Important jobs can receive higher priority without allowing them to consume the entire cluster.

5. Monitor Resources

Monitor:

- CPU usage
- Memory usage
- Queue utilization
- Running applications

Conclusion

The resource starvation is mainly caused by **poor YARN scheduling and resource allocation**. Queue-based resource management, fairness, priorities, and limits can provide better resource utilization.

5. Backup Restores Outdated Data

Problem

After a system failure, the backup restores an old version of the data instead of the most recent version.

Likely Design Flaws

1. Infrequent Backups

Backups may be performed too rarely.

2. Outdated Replicas

The backup copy may not be synchronized with the primary data.

3. Incorrect Backup Selection

The recovery process may select an older backup instead of the latest valid backup.

4. Missing Backup Verification

Backups may not be tested regularly.

5. Replication ≠ Backup

Replication provides additional copies, but if data is deleted or corrupted and that change is replicated, all copies may be affected.

Corrective Actions

- Schedule backups more frequently.
- Use incremental backups.
- Maintain multiple backup versions.
- Use timestamps/version information.
- Maintain geographically separate backup copies.
- Verify backup integrity.
- Perform regular recovery tests.
- Define **RPO (Recovery Point Objective)** according to business requirements.
- Use replication together with independent backups.

Example

If backups are taken only once every 24 hours, a failure at 10 PM could cause the organization to lose up to nearly a day's changes.

A shorter backup interval reduces this potential loss.

Conclusion

The main design flaw is **poor backup frequency or synchronization and inadequate recovery validation**. Versioned backups, frequent snapshots, replication, and regular recovery testing ensure that the latest usable data can be restored.